Wave 1 Execution Playbook

Denai Clinical Platform — Safe Modularization Governance

Classification: Execution Governance — Deterministic Preservation

---

Preamble

This playbook governs the first wave of safe module extraction from the denai clinical platform. It is a binding execution document. Every rule in this playbook reflects a specific, documented risk from the completed synchronization analysis, render trigger chain analysis, architecture risk consolidation, and DOM lifecycle audit.

Wave 1 is bounded exclusively to code that is demonstrably pure: no DOM ownership, no render orchestration, no state mutation, no timer registration, no observer registration, and no implicit coupling to the global state object `S` or the `tempState` shadow. Any extraction target that cannot be verified clean against all five of these dimensions is out of scope for Wave 1 and must be deferred.

This wave does not modularize the application. It extracts self-contained units from it. The application's internal wiring, sequencing, timing, and orchestration are not touched.

---

1. Wave Objectives

Primary objective: Isolate and externalize all pure, stateless code units — pure helpers, validators, constants, formatting utilities, and non-mutating calculation helpers — without altering any runtime behavior of the production application.

Secondary objective: Establish the extraction protocol and verification infrastructure that all subsequent waves (Wave 2+) will build on. Wave 1 sets the standard of evidence required before any code moves.

Explicit non-objectives:

- Do not improve, refactor, or optimize any extracted unit
- Do not rename functions or variables beyond the minimum required by module export

syntax

- Do not consolidate extracted units into logical groupings that did not exist before
- Do not introduce new abstractions, interfaces, or wrappers
- Do not alter call sites inside the main application beyond the import replacement
- Do not add error handling to extracted units
- Do not add tests to the main application beyond the regression baseline required by this playbook

Success criteria:

- Zero behavior change in any observable output of the application after extraction
- Zero timing changes in any async path
- Zero DOM mutation changes
- Zero changes to `S` state sequencing
- All rollback checkpoints verified clean before proceeding to the next extraction unit

---

2. Allowed Extraction Targets

An extraction target is allowed in Wave 1 if and only if it satisfies ALL of the following conditions simultaneously:

2.1 Purity requirements (all must be true)

- The function or constant accepts inputs and returns outputs with no side effects
- It does not read from `S`, `tempState`, or any module-scoped variable outside its own file
- It does not write to `S`, `tempState`, or any module-scoped variable
- It does not call `setState()`, `saveState()`, `render()`, or any function with render responsibilities
- It does not call `renderPatientList()`, `renderMainPanels()`, `renderCompoundSummary()`, `renderTxCards()`, `renderComparison()`, `renderMaterial()`, `renderCost()`, `renderGraph()`, `renderRisk()`, `renderReasons()`, `renderPatientDisplay()`, or `History.render()`
- It does not reference `document`, `window`, `localStorage`, `sessionStorage`, or any

browser API

- It does not register or clear any `setTimeout`, `setInterval`, `requestAnimationFrame`, or `IntersectionObserver`
- It does not import from, or call into, any function that violates any of the above

2.2 Isolation requirements (all must be true)

- It can be copied to a new file and executed in a pure Node.js environment with no mocks
- All of its dependencies are either (a) native JavaScript, (b) already-extracted Wave 1 modules, or (c) inline constants within the same extraction unit
- It has no circular dependency with any function that owns DOM state

2.3 Confirmed allowed target categories

- Named constants and configuration objects (`BRAND`, `DEFAULT_STATE` field definitions — see Section 3 for forbidden portions of `DEFAULT_STATE`)
- Pure string formatting functions (name formatting, label formatting, date formatting)
- Pure number formatting functions (score formatting, percentage formatting, cost display formatting)
- Pure validation functions that return a boolean or validation result without mutating inputs
- Pure calculation helpers: functions that accept clinical parameters as arguments and return a computed score or classification, with no access to `S`
- `StateValidator.validate(field, value)` if and only if it is a pure function that does not mutate `S` and does not trigger any render
- Pure classification functions (e.g., tooth type classification, condition category mapping)
- Pure recommendation label maps and lookup tables
- Pure cost computation helpers (`computeCosts(state, ai)` — eligible only if verified to be pure; see Section 4)
- Pure material selection helpers (`getCrownMaterial(state)` — eligible only if verified to be pure)
- `DOMPurify.sanitize` wrapper if one exists as a local wrapper function — the wrapper only, not the library
- The `MULTI_TO_SINGLE` treatment mapping object

- The `PRESETS` constants object
- Pure score normalization functions

---

3. Forbidden Extraction Targets

The following are explicitly out of scope for Wave 1. Each prohibition is grounded in a specific documented risk.

3.1 Absolutely forbidden — carry orchestration or DOM ownership

- `render(S)` — root orchestrator; forbidden at all waves until all dependencies are extracted
- `renderMainPanels()` — carries withErrorBoundary, typewriter, material timer, SVG two-pass
- `buildAICardStructure()` — DOM owner with `dataset.built` flag; structural rebuild contract
- `updateAICard()` / `updateAICardMulti()` — patch owners; `_typewriterTimer` registration
- `renderComparisonTable()` / `lazyRenderComparisonTable()` — observer registration, closure capture
- `renderCompoundSummary()` — compound branch DOM owner
- `renderPatientList()` — sidebar DOM owner; side effect of saveState
- `renderMaterial()` — `_matFadeTimer` owner
- `renderTxCards()` — load-bearing ordering dependency with `updateCrownCardState()`
- `updateCrownCardState()` — reads DOM `.classList` state written by `renderTxCards()`
- `renderRisk()`, `renderReasons()`, `renderCost()`, `renderGraph()`, `renderPatientDisplay()` — DOM writers

3.2 Absolutely forbidden — carry state mutation

- `setState()` — state validator and diff tracker; mutation entry point
- `saveState()` — mutates persistence, triggers `renderPatientList()`, owns `_saveStateTimer`

- `switchPatient()` — full S replacement, deferred render, cross-patient corruption window
- `selectTx()` — direct `S.tx` mutation + render trigger
- `applyAIRec()` — direct `S.tx` mutation, bypasses `setState()`
- `toggleMultiSite()` — direct `S.multiSite` mutation, pre-wipe of DOM, render trigger
- `whatIfApply` click handler — `Object.assign(S, ...)`, bypasses setState, render trigger
- `applyPreset()` — `Object.assign(tempState, ...)`, direct updatePreview call
- `cancelEdit()` / `exitEdit()` — DOM and state mutation
- `saveEdit()` — `setState()` call + render chain
- `buildEditForm()` — DOM owner (innerHTML of `#editFields`)
- `initClinicalNotes()` — listener binding with `_notesInputBound` guard, `S.notes` mutation
- `updateSliderPositions()` — DOM writes to slider elements from tempState/S
- `updatePreview()` / `debouncedUpdatePreview()` — timer owner, partial render path, `_isPreviewMode` mutation
- `showSkeleton()` — DOM owner; clears `dataset.built`
- `generateReport()` — report button mutation, orphaned 800ms timer, URL allocation
- `History.add()` / `History.render()` / `History.save()` / `History.load()` — persistence + DOM owner chain

3.3 Absolutely forbidden — carry timer or observer registration

- `typewriterEffect()` — registers `_typewriterTimer` (setInterval, 30ms); module-scoped singleton
- Any function wrapping `CleanupRegistry.timer()` calls
- `_compTableObserver` setup logic

3.4 Forbidden portions of otherwise-eligible modules

- `DEFAULT_STATE` as a complete object: forbidden if it contains UI-transient flags (`editing`, `whyOpen`, `historyOpen`). Only the clinical field defaults may be extracted; the transient flags must remain in-place to preserve the existing destructure-blocklist

serialization pattern.

- `ClinicalEngine.process()` and `ClinicalEngine.processCompound()` : forbidden. These are eligible for Wave 2 only after their internal pure sub-functions are extracted individually in Wave 1. The routing logic inside `process()` (`MISSING_SINGLE` , `MISSING_MULTI` , `RESTORATIVE` branch) is load-bearing and must not be separated from its callers prematurely.
- `calcAI()` and `calcAIMulti()` : forbidden. `calcAI()` is called by `applyAIRec()` and `whatIfApply` directly, bypassing `ClinicalEngine.process()` . These call sites carry undocumented behavioral coupling. Extraction without resolving those call sites first would require modifying the call sites — which is out of scope.
- `withErrorBoundary()` : forbidden. It wraps DOM container IDs and performs innerHTML replacement on failure. It is not a pure utility.
- `showToast()` : conditional. If `showToast()` is a pure function that only constructs and appends a toast element without reading from `S` , it is eligible. If it reads from `S` or has any implicit state dependency, it is forbidden. Must be individually verified.
- `StateValidator` : if `StateValidator.validate()` is a pure function, only the validation logic is eligible. If the validator has a side-channel write (e.g., logging to a module-scoped array, emitting an event), it is forbidden.
- `genCaseNum()` : forbidden. It reads from `loadAllPatients()` (localStorage). It is not pure.

---

4. Mandatory Pre-Extraction Checks

Before any extraction begins on any individual target, the following checks must be completed and documented. Every check must produce a binary pass/fail result. Any single fail halts extraction of that target.

4.1 Static analysis checks (manual, required before touching any file)

- **S-read scan:** Search the candidate function body for any reference to `S` , `tempState` , `window.S` , or any alias for the global state object. Any hit = FAIL.
- **S-write scan:** Search for `S.` , `Object.assign(S` , `setState(` , `S[` . Any hit = FAIL.
- **DOM API scan:** Search for `document.` , `getElementById` , `querySelector` , `innerHTML` , `classList` , `style.` , `setAttribute` , `dataset.` . Any hit = FAIL.

- **Browser API scan:** Search for `window.`, `localStorage.`, `sessionStorage.`, `navigator.`, `location.`. Any hit = FAIL.
- **Timer scan:** Search for `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`, `requestAnimationFrame`, `IntersectionObserver`. Any hit = FAIL.
- **Render call scan:** Search for any call to render functions listed in Section 3.1. Any hit = FAIL.
- **Import chain scan:** List every function called by the candidate. For each dependency, repeat the above scans. A dependency that fails any scan = candidate FAIL.
- **Module-scope variable scan:** Check if the candidate reads or writes any module-scoped variable (not a parameter, not a local variable, not an imported constant). Any mutable module-scope variable access = FAIL.

4.2 Behavioral equivalence baseline (required before extraction)

- Record the exact output of the candidate function for a minimum of three representative inputs covering: (a) a normal clinical case, (b) an edge-case or missing-data case, (c) a multi-site or multi-tooth case where applicable.
- Record the output as a frozen assertion. This is the equivalence baseline. It must be reproduced identically after extraction.
- If the candidate function's output depends on any external data source (beyond its parameters), document that dependency explicitly. If the dependency is not passable as a parameter, the function is not pure and is FORBIDDEN.

4.3 Call site inventory (required before extraction)

- List every location in the codebase where the candidate function is called.
- For each call site, record: the calling function name, the arguments passed, and the context in which it is called (synchronous render path, async timer callback, direct user event, etc.).
- Verify that every call site passes all required inputs as explicit arguments (no implicit global reads at the call site that would become implicit module-scope reads after extraction).
- If any call site passes `S` or `tempState` as an argument: this is permitted (the extracted function receives state as a parameter but does not import or close over it). Document this explicitly.
- If any call site relies on the function having been called in a specific prior sequence (order dependency): this is a hidden coupling. Log it in the coupling detection checklist

(Section 5) and evaluate whether the order dependency can be safely preserved after extraction.

---

5. Hidden Coupling Detection Checklist

These are the coupling patterns documented in the prior analyses that must be actively checked for in every Wave 1 candidate. Each is a class of hidden dependency that pure-function appearance does not rule out.

5.1 Shared module-scope singleton access

- Does the candidate read `_typewriterTimer`, `_matFadeTimer`, `_saveStateTimer`, `_notesDebounce`, `_compTableObserver`, `_lastSimAI`, `_isPreviewMode`, `_notePatient`, `_notesInputBound`?
- If yes: the function is coupled to the async runtime and is FORBIDDEN.

5.2 ClinicalEngine routing assumption

- Does the candidate assume it will only be called from a specific branch of `ClinicalEngine.process()`?
- If yes: it carries an implicit precondition that extraction would silently break. Document the precondition. It is FORBIDDEN unless the precondition can be expressed as an explicit parameter.

5.3 calcAI vs. ClinicalEngine divergence

- Does the candidate produce output that is only correct when consumed by `calcAI()` and NOT by `ClinicalEngine.process()`, or vice versa?
- The documented risk: `calcAI()` and `ClinicalEngine.process()` produce different recommendation objects for restorative conditions. Any helper that assumes the structure of one and is called from the context of the other carries a silent semantic error.
- If yes: document the assumed caller context and mark as requiring human review before extraction.

5.4 DOM-read precondition

- Does the candidate assume that a specific DOM element exists or has a specific state (e.g., a `.classList` value written by a prior render call)?
- Documented risk: `updateCrownCardState()` reads `.classList` set by

`renderTxCards()`. Any helper that reads DOM state is not pure.

- If yes: FORBIDDEN.

5.5 Ordering contract dependency

- Does the candidate's correctness depend on having been called after another function in the same render cycle?
- Documented risk: `buildAICardStructure()` must precede `updateAICard()`. Pure helpers should not carry such dependencies, but verify explicitly.
- If yes: document the dependency. If the dependency cannot be encoded as a parameter, FORBIDDEN.

5.6 tempState selective promotion assumption

- Does the candidate operate on fields that are included in the `whatIfApply` selective promotion mapping (`bone`, `hygiene`, `occlusion`, `smoking`, `diabetes`, `remainingStructure`, `endodonticStatus`, `parafunction`) versus fields that are intentionally excluded (`tooth`, `name`, `age`, `condition`, `tx`, `gender`)?
- If the candidate blends both field categories in a way that assumes the promotion boundary: document explicitly. If extraction would break that boundary assumption at any call site: FORBIDDEN.

5.7 Transient UI flag leakage

- Does the candidate reference or return values that include `editing`, `whyOpen`, or `historyOpen`?
- These flags are excluded from persistence via the destructure-blocklist pattern. Any extracted module that handles these flags without preserving the blocklist pattern would alter persistence behavior.
- If yes: FORBIDDEN.

5.8 StateDiffs side-channel

- Does the candidate, anywhere in its call chain, push to a `StateDiffs` array or equivalent diff-tracking structure?
- If yes: FORBIDDEN. The diff-tracking is a side effect tied to `setState()` orchestration.

5.9 Preview mode delta baseline

- Does the candidate interact with `_lastSimAI` or compute against the `_previewScore` field that is mutated onto the `ai` object inside `updatePreview()`?

- Documented risk: `ai._previewScore` is mutated directly onto the object returned from `ClinicalEngine.process()`. Any helper that reads or writes this field is coupled to the preview runtime.
- If yes: FORBIDDEN.

---

6. Safe Extraction Sequence

Extractions must proceed in this order. No step may begin before the previous step's rollback checkpoint has been verified. Steps within the same tier may be extracted in parallel only if they have no shared dependencies.

Tier 0 — Baseline establishment (no code changes)

1. Freeze a behavioral baseline: run the application end-to-end and record outputs for a representative set of clinical cases covering single-site, multi-tooth, multi-site compound, restorative, and missing-tooth conditions. This baseline is the regression reference for all subsequent steps.
2. Create a rollback checkpoint: commit the current codebase in its unmodified state. Tag this commit `wave1-baseline`. This tag must not be deleted.

Tier 1 — Constants and lookup tables (lowest risk) 3. Extract pure constant objects: `BRAND`, `PRESETS`, `MULTI_TO_SINGLE` map, treatment label maps, tooth classification maps, condition category maps. 4. Extract field-level default value constants (clinical fields only — exclude transient UI flags from `DEFAULT_STATE`). 5. Rollback checkpoint after Tier 1.

Tier 2 — Pure string and number formatters 6. Extract pure string formatting helpers: name formatting, label construction, badge text generation. 7. Extract pure number formatting helpers: score display formatting, percentage formatting, cost display formatting. 8. Rollback checkpoint after Tier 2.

Tier 3 — Pure validators 9. Extract `StateValidator.validate()` if and only if it passes all pre-extraction checks. Extract the validation logic only — not any wrapper that calls `setState()` or `StateDiffs`. 10. Extract any standalone field-level validation functions (range checks, allowed-value sets, type checks). 11. Rollback checkpoint after Tier 3.

Tier 4 — Pure clinical calculation helpers 12. Extract individual pure sub-functions from within `ClinicalEngine` (score normalization, risk classification, scoring weights). Extract these as standalone functions, NOT as a refactored `ClinicalEngine` module. 13. Extract `computeCosts(state, ai)` if verified pure. Extract `getCrownMaterial(state)` if verified pure. 14. Extract pure recommendation mapping and classification functions. 15. Rollback

checkpoint after Tier 4.

Tier 5 — Conditional extractions (require individual human review) 16. `showToast()` — if and only if it passes all pre-extraction checks and human review confirms no implicit state dependency. 17. Any remaining pure helpers identified during Tier 1–4 work that were not initially catalogued. 18. Rollback checkpoint after Tier 5.

Wave 1 close 19. Full regression run against the Tier 0 behavioral baseline. 20. Confirm all extracted modules pass isolation tests (Section 8). 21. Tag the final commit `wave1-complete`.

---

7. Import/Export Safety Rules

7.1 Export rules

- All Wave 1 extractions use named exports only. No default exports.
- Export only the final public interface of each module — do not export internal helper sub-functions unless those sub-functions are themselves standalone extraction targets.
- Do not add re-exports in `index.js` or barrel files during Wave 1. Each module is imported directly by its consumers. Barrel files are a Wave 3+ concern.
- Do not add JSDoc, TypeScript types, or interface definitions during extraction. Documentation is not part of Wave 1 scope.

7.2 Import rules

- The only allowed imports in a Wave 1 module are: (a) other Wave 1 modules already extracted in a prior tier, (b) npm packages that were already in use in the original code (no new dependencies), and (c) no imports at all (preferred for Tier 1 constants and formatters).
- No Wave 1 module may import from the main application file or from any file that contains `render`, `setState`, `saveState`, `S`, or `tempState`.
- Circular imports are forbidden. Verify with a dependency graph trace before each extraction.

7.3 Call site replacement rules

- After extracting a function, replace every call site in the main application with the import. Do not leave duplicate copies of the function — one in the module, one inline in the application.

- The import statement must be placed at the top of the main application file, in a designated Wave 1 imports section, grouped separately from existing imports.
- The replacement at the call site must be syntactically identical to the original call (same function name, same argument order, same return value consumption). If the function was previously a local declaration and is now an import, the name must not change.
- If a call site passes `S` or `tempState` as an argument to the extracted function, this pattern must be preserved exactly. The extracted function receives state as a parameter; it does not import or close over it.

7.4 Forbidden import patterns

- Do not import `S`, `tempState`, or any module-scoped runtime variable into any Wave 1 module.
- Do not import `render`, `renderMainPanels`, or any render function into any Wave 1 module.
- Do not import `setState`, `saveState`, or any persistence function into any Wave 1 module.
- Do not import `CleanupRegistry` or any timer management utility into any Wave 1 module.

---

8. Validation Requirements After Every Extraction

Every extraction — including single-function extractions — must pass all of the following validations before the next extraction begins. No exceptions.

8.1 Isolation validation

- The extracted module must execute in isolation in a pure Node.js environment (`node module.js`) with no application context, no DOM, no browser APIs, no `S`, and no `tempState`. If it throws or requires mocking to run, it fails isolation validation.
- Run the behavioral equivalence baseline cases (from Section 4.2) against the isolated module. Outputs must be byte-for-byte identical to the recorded baseline.

8.2 Call site validation

- Every call site identified in the pre-extraction call site inventory must be verified to call the imported version with no behavior change.
- If the application has any automated tests, all must pass. If not, the behavioral baseline

must be manually re-verified for the specific user paths that exercise the extracted function.

8.3 No-new-import validation

- Verify that the main application file's module-scope variable list, timer registry, and observer registry are unchanged after extraction. The extraction must not have introduced any new closures, new module-scope variables, or new async registrations.
- Verify that the extracted module's import chain resolves with no new npm packages.

8.4 Timing validation

- For any extraction in Tier 4 (clinical calculation helpers), manually verify that the execution of `typewriterEffect`, `_matFadeTimer`, and `_saveStateTimer` are unaffected. Time the material render 160ms window before and after extraction. If the window changes by any measurable amount, the extraction has introduced a timing side effect and must be rolled back.

8.5 DOM state validation

- After any Tier 4 extraction, load the application and perform the following sequence: (a) load a patient, (b) switch treatment, (c) switch patient, (d) enable multi-site mode, (e) move a simulator slider. At each step, verify that the DOM state (AI card content, comparison table, material display, SVG tooth highlight) is identical to the pre-extraction baseline.

---

9. Rollback Checkpoint Protocol

9.1 Checkpoint creation

- A rollback checkpoint is a git commit tagged with `wave1-ckpt-{tier}-{n}` where `{tier}` is the current tier number and `{n}` is a sequential counter within that tier.
- Checkpoints are created: (a) before any extraction begins on a new target, (b) after each successful extraction and validation, and (c) at the explicit end of each tier.
- The checkpoint commit message must include: the name of the extracted module, the call sites modified, and the validation results (pass/fail for each check in Section 8).

9.2 Rollback trigger conditions Roll back to the most recent checkpoint immediately if any of the following occur:

- Any validation in Section 8 fails after extraction

- Any red flag from Section 12 is observed during or after extraction
- Any unexpected import is required to make the extraction work
- The application throws any new exception after extraction
- Any timer fires at a different time than the pre-extraction baseline (measurable by stopwatch against the 160ms material window or 300ms patient switch window)
- The `dataset.built` flag behavior on `#aiCardBody` changes in any detectable way
- The comparison table renders with different content than the pre-extraction baseline for the same input state
- Any previously-passing manual test case fails

9.3 Rollback execution

- `git checkout wave1-ckpt-{most-recent-passing-checkpoint}`
- Do NOT attempt to fix the failed extraction in place. Roll back first, understand the failure, then re-plan.
- After rollback, the failed extraction target must be re-classified: either (a) confirmed forbidden and added to Section 3, or (b) placed in a deferred list for human review before re-attempting.

9.4 Checkpoint archive

- All checkpoint tags must be retained for the duration of the Wave 1 extraction and for 30 days after `wave1-complete` is tagged.
- Checkpoint tags must not be force-pushed over or deleted during this window.

---

10. Regression Testing Protocol

10.1 Behavioral baseline cases (mandatory, defined at Tier 0)

The following test cases must be manually executed and recorded before any extraction begins, and re-executed after every tier completion:

Case ID	Scenario	Panels to verify
R01	Single-site missing tooth, implant selected	AI card text, ring SVG, comparison table, material, cost

R02	Single-site restorative, crown selected, crown viable	AI card, treatment cards, comparison table, cost, material
R03	Single-site restorative, crown selected, crown NOT viable	Toast error, no treatment change
R04	Multi-tooth mode, implant + bridge	AI card multi layout, comparison table column schema
R05	Multi-site compound mode, both sites configured	Compound summary, SVG two-pass highlights, tab labels
R06	Patient switch during active preview mode	Preview banner retention on new patient (stale — expected)
R07	Slider move → treatment card click within 120ms	Treatment card click wins; preview does NOT overwrite
R08	Slider move → preset apply within 120ms	Preset result visible; slider debounce may overwrite after 120ms (expected behavior)
R09	Notes typed → patient switch before 800ms debounce	Note content not saved to new patient (expected — silent abort)
R10	Report generation in preview mode	Report reflects S, not tempState; no warning (expected behavior)
R11	Comparison table offscreen → scroll into view after state change	Observer fires with closure state; may be stale vs. current (expected behavior)
R12	Rapid patient switch (two switches within 300ms window)	Verify no cross-patient data corruption in localStorage

10.2 Timing-sensitive case verification

These cases require timing observation, not just output comparison:

Case ID	Scenario	What to time	Expected duration
T01	Render with material change	Time from render call to `#matPrimary` content update	160ms ± 20ms
T02	Patient switch	Time from click to full panel update	300ms ± 30ms

T03	Notes autosave	Time from typing stop to `S.notes` mutation	800ms ± 50ms
T04	State autosave	Time from `setState()` to `renderPatientList()` fire	200ms ± 20ms
T05	Typewriter animation	Time per character append	30ms ± 5ms

10.3 Regression pass criteria

- All R-series cases: outputs must be byte-for-byte identical to Tier 0 baseline
- All T-series cases: timings must be within the stated tolerance
- Any deviation — even expected behavior that now manifests differently — is a regression and must be investigated before proceeding

---

11. Deterministic Preservation Rules

These rules encode the specific behavioral contracts from the prior analyses that must survive Wave 1 extraction unchanged.

11.1 Two-pass SVG compound highlight No extraction in Wave 1 may touch `updateToothHighlight`, `updateToothHighlight2`, or the render logic that calls them. The second-pass overwrite inside `render()` — which overwrites the first-pass SVG state set inside `renderMainPanels()` — is a load-bearing architectural contract. Wave 1 extractions must not move, inline, or abstract any code in this path.

11.2 dataset.built flag integrity The `dataset.built === '1'` gate on `#aiCardBody` must remain the sole determinant of structural vs. patch render for the AI card. No Wave 1 extraction may introduce a secondary gate, remove this gate, or move the flag-setting logic.

11.3 withErrorBoundary terminal state The behavior where a `withErrorBoundary` catch replaces a container's innerHTML with an error card — and subsequent render attempts silently fail against the error markup — must be preserved as-is. No extraction may introduce automatic error recovery.

11.4 saveState → renderPatientList side-effect chain The chain `saveState()` → 200ms debounce → `renderPatientList()` must remain intact. This is a hidden orchestration side effect. If any extracted Tier 4 helper previously shared a code path with `saveState()`, verify that the extracted version does not alter the debounce registration or the `renderPatientList()` call.

11.5 Observer closure capture The `_compTableObserver` captures `state` and `ai` at creation time. This closure-based stale capture is the existing behavior. No extraction may introduce a mechanism that refreshes the observer's captured values.

11.6 Material 160ms window The 160ms gap between `renderMaterial()` call and `#matPrimary` DOM write is a documented behavior that downstream timing relies on. No extraction may alter the 160ms value or convert the timer to a synchronous write.

11.7 Notes autosave silent abort The behavior where the 800ms notes debounce silently aborts if `S.id !== _notePatient` must remain intact. If `_notePatient` capture is adjacent to an extraction target, verify it is not disturbed.

11.8 calcAI vs. ClinicalEngine divergence The behavioral divergence between `calcAI()` and `ClinicalEngine.process()` for restorative conditions is an existing condition. No Wave 1 extraction may inadvertently unify these paths or introduce a shared helper that changes either output.

11.9 Transient flag blocklist The destructure pattern `const { editing, whyOpen, historyOpen, ...serializable } = S` in the `saveState()` timer callback defines which fields are excluded from persistence. This is a blocklist pattern. Any extraction of constants that includes these field names must not alter their presence in this blocklist.

11.10 selectTx early-exit branches The `selectTx()` function has an early-exit branch for `S.multiTooth` that skips `History.add()` and the toast. If any pure helper extracted from within `selectTx()`'s call chain is used elsewhere, it must not carry the assumption that `History.add()` has or has not been called.

---

12. Red Flags That Require Stopping Extraction

Stop all extraction activity immediately and do not proceed until the flag is investigated and resolved:

RF-01: Unexpected import required The extraction of a target requires importing something that was not listed in the pre-extraction call site inventory. This means the static analysis was incomplete and hidden dependencies exist.

RF-02: Module-scope variable discovered mid-extraction During extraction, a module-scoped mutable variable is discovered inside the candidate that was not visible from the function signature. The function has an implicit stateful dependency.

RF-03: Test case output changes Any R-series regression case produces different output after extraction, even if the difference appears cosmetically minor (whitespace in a

generated string, a different score decimal, a different label text). All differences are significant.

RF-04: Timer timing changes Any T-series timing case falls outside its tolerance window after extraction.

RF-05: calcAI / ClinicalEngine output diverges further After extraction of any clinical calculation helper, running R01 and R02 against both `calcAI(S)` and `ClinicalEngine.process(S)` produces a difference that did not exist in the Tier 0 baseline. The extraction has altered one of the two paths.

RF-06: dataset.built behaves differently After any extraction, the AI card does not correctly distinguish between structural rebuild and patch render. Any extra rebuild, any skipped rebuild, or any error-fallback that behaves differently than the Tier 0 baseline.

RF-07: Comparison table content mismatch After any Tier 4 extraction, the comparison table shows different row values than the Tier 0 baseline for the same input state, even if the table renders at the same time.

RF-08: Cross-patient data write detected At any point during Wave 1 validation, localStorage inspection reveals a patient record that has been overwritten with another patient's field values. This is the save-after-switch corruption pattern. Stop all work. This is not caused by Wave 1 extraction (Wave 1 does not touch the save path), but its presence indicates the environment is not safe for extraction work.

RF-09: New async registration appears After any extraction, the browser's task queue (visible via performance profiler) shows a new timer or observer registration that did not exist in the Tier 0 baseline. The extraction has introduced a new async path.

RF-10: `s` appears in an extracted module's scope After extraction, any linting pass, grep, or manual review finds a reference to `S` (the global state object) in the extracted module file. Stop. The extraction is contaminated.

---

13. Conditions Requiring Escalation to Claude Opus

Escalate to Claude Opus (full-context analysis) before proceeding when:

E-01: Hidden coupling discovered but nature is unclear A static analysis scan reveals a potential coupling (e.g., a function that calls another function that calls another function, and somewhere deep in the chain there is a `S.` reference) but the coupling path cannot be fully traced within the current analysis context.

E-02: ClinicalEngine sub-function boundary is ambiguous When extracting a sub-

function from within `ClinicalEngine`, the boundary between the pure sub-function and the routing logic (`MISSING_SINGLE`, `MISSING_MULTI`, `RESTORATIVE` branch) is not clear enough to extract safely without risk of altering the branching behavior.

E-03: calcAI purity is ambiguous `calcAI()` or `calcAIMulti()` appears to be pure but has call site dependencies (it is called from `applyAIRec()` bypassing `ClinicalEngine.process()`, and from `whatIfApply` directly) that create ambiguity about whether extracting its internals would alter the semantic difference between `calcAI` and `ClinicalEngine.process()`.

E-04: StateValidator has side channels `StateValidator.validate()` is discovered to push to a log, emit an event, or write to any data structure beyond its return value. Its extraction scope must be re-defined by a full analysis pass before proceeding.

E-05: Timing regression cannot be explained A T-series timing case falls outside tolerance and the cause is not immediately identifiable from the extraction changeset. Do not attempt to fix it. Escalate for full timing path analysis.

E-06: Multiple extraction targets share an undocumented shared closure Two or more candidates that appeared independent are found to share a closure over a module-scoped variable that was not caught by the static analysis. The scope and impact of the shared closure must be fully mapped before either target is extracted.

E-07: A Wave 1 extraction reveals a new hidden orchestration dependency During the extraction process, evidence is found of an orchestration dependency not documented in the prior analyses. This dependency must be fully characterized before extraction continues, as it may affect Wave 2 and beyond scope planning.

---

14. Conditions Requiring Human Review Before Apply

The following require a human engineer to review the extraction diff and explicitly approve before the change is applied to any tracked branch:

H-01: Any extraction touching clinical calculation logic Any Tier 4 extraction that involves scoring weights, recommendation generation logic, risk classification, or treatment selection logic requires human clinical engineering review. Incorrect extraction of clinical logic is a patient-safety concern.

H-02: DEFAULT_STATE field extraction Any extraction of default state values requires human review to confirm the transient flag exclusion is correct and complete. The blocklist pattern (`editing`, `whyOpen`, `historyOpen`) must be verified by a human who understands the full persistence schema.

H-03: StateValidator extraction Human review required to confirm the extracted validator does not carry any side-channel behavior (diff tracking, logging, event emission).

H-04: showToast() extraction Human review required to confirm `showToast()` has no implicit dependency on `S` or on any render sequencing assumption.

H-05: Any extraction that modifies more than one call site If a single extraction requires modifying more than one location in the main application file, human review is required to verify that all call sites are equivalent replacements.

H-06: Any extraction where the candidate was previously called from inside a withErrorBoundary closure The `withErrorBoundary` pattern groups functions that share an error recovery contract. Extracting a function from inside this grouping must be reviewed to confirm the error recovery behavior is not altered.

H-07: Rollback after a failed extraction Any time a rollback is executed (per Section 9.3), human review is required before re-attempting the failed target. The failure reason must be documented and the re-extraction plan must be approved by a human before execution.

---

15. Claude Code Execution Constraints

When using Claude Code to assist with Wave 1 extraction, the following constraints are binding:

15.1 Allowed operations

- Reading source files for static analysis
- Searching for function call patterns, variable references, and import chains
- Generating the content of the extracted module file (not applying it)
- Generating the import statement to be added to the main application
- Generating the call site replacement (not applying it)
- Running isolation tests against extracted modules in a pure Node.js context

15.2 Forbidden operations

- Claude Code must not directly apply changes to the main application file without a human-confirmed review step
- Claude Code must not modify `render()`, `renderMainPanels()`, `saveState()`, `setState()`, `switchPatient()`, or any function in the forbidden list (Section 3)

- Claude Code must not add imports to the main application file automatically — imports must be reviewed and applied manually
- Claude Code must not run the application in a browser context during extraction (side effects from timer execution could corrupt baseline measurements)
- Claude Code must not delete any file from the repository
- Claude Code must not reorder code within any file (even if the extraction leaves whitespace or gaps — leave them)
- Claude Code must not create barrel/index files or reorganize the directory structure

15.3 Maximum autonomy per session

- Claude Code may autonomously extract and validate a maximum of one Tier 1 or Tier 2 target per session
- All Tier 3, Tier 4, and Tier 5 extractions require explicit human confirmation at each step before Claude Code proceeds
- Claude Code must stop and report after each validation step, not proceed to the next extraction automatically

15.4 Output format requirements

- All generated extraction files must be presented as diffs or new-file contents for human inspection before write
- All call site replacements must be presented as explicit before/after pairs, not as in-place edits
- All validation results must be logged as a structured checklist (Section 8 format) before the next step

---

16. Maximum Allowed Extraction Scope Per Iteration

An iteration is defined as one Claude Code session or one human engineering work session (not to exceed 4 hours).

Tier 1 iterations: Maximum 5 constant objects or lookup tables per iteration. **Tier 2 iterations:** Maximum 3 formatter functions per iteration. **Tier 3 iterations:** Maximum 2 validator functions per iteration. **Tier 4 iterations:** Maximum 1 clinical calculation helper per iteration. **Tier 5 iterations:** Maximum 1 conditional extraction per iteration.

These limits are not performance targets. They are safety ceilings. Extracting less than the

maximum is always preferred. The limits exist to ensure that rollback scope is bounded — if an extraction fails, the maximum damage is the loss of one iteration's work, not a multi-function extraction that is difficult to untangle.

If a single target proves to require more changes than anticipated (more call sites than inventoried, more dependencies than catalogued), stop the iteration, log the finding, and reschedule. Do not expand scope mid-iteration to compensate.

---

17. Required Review-Before-Apply Workflow

Every extraction follows this exact sequence. Deviating from the sequence — including skipping steps that seem unnecessary for "obviously safe" extractions — is forbidden.

```
Step 1: IDENTIFY
  └─ Name the target. Document why it is in scope (which allowed category, Sectio

Step 2: PRE-CHECK
  └─ Complete all checks in Section 4. Record binary pass/fail for each.
  └─ Complete all coupling checks in Section 5. Record findings.
  └─ If any fail: reclassify target as forbidden. Stop.

Step 3: BASELINE
  └─ Record behavioral equivalence baseline (Section 4.2).
  └─ Record call site inventory (Section 4.3).

Step 4: GENERATE
  └─ Generate extracted module content. Do not apply.
  └─ Generate import statement. Do not apply.
  └─ Generate call site replacements. Do not apply.
  └─ Present all three as a diff for human review.

Step 5: HUMAN REVIEW
  └─ Engineer reviews the diff.
  └─ Check if any H-series condition (Section 14) applies. If yes: full H-series
  └─ Engineer explicitly approves or rejects.
  └─ If rejected: log reason, reclassify target. Stop.

Step 6: CHECKPOINT
  └─ Create rollback checkpoint BEFORE applying any changes (Section 9.1).

Step 7: APPLY
  └─ Write extracted module file.
  └─ Add import statement to main application.
  └─ Replace call sites (one at a time, verifying each).
```

```
Step 8: VALIDATE
  └─ Run all Section 8 validations. Record results.
  └─ If any validation fails: execute rollback (Section 9.2–9.3). Stop.

Step 9: CHECKPOINT
  └─ Create rollback checkpoint AFTER successful validation.

Step 10: LOG
  └─ Document extraction in the Wave 1 extraction log:
      - Target name
      - File location before and after
      - Call sites modified
      - Validation results
      - Any coupling findings
      - Any human review decisions
```

---

18. Dependency Verification Requirements

Before any extraction, the full dependency chain of the candidate must be traced. The following evidence must be on record:

18.1 Direct dependencies List every function, constant, and external module referenced in the candidate. For each: (a) confirm it is pure or an already-extracted Wave 1 module, and (b) confirm it will be available in the extracted module's import scope without importing from the main application.

18.2 Indirect dependencies (depth-2 scan) For each direct dependency, repeat the dependency scan. If any depth-2 dependency violates Section 2 purity requirements, the original candidate is contaminated.

18.3 Reverse dependency scan List every function in the codebase that depends on the candidate. This is required to ensure the call site inventory is complete. Any call site not in this list is a missed replacement location.

18.4 Circular dependency check Trace the complete import graph from the candidate's proposed new file location. Confirm there is no cycle. A cycle at any depth is a FAIL.

18.5 npm package dependency check If the candidate uses any npm package, confirm: (a) the package is already in `package.json` (no new dependencies), and (b) the package is side-effect free (does not register globals, mutate prototypes, or register browser APIs on import).

---

19. Naming and File-Boundary Rules

19.1 File naming

- Extracted module files must be named after their primary content, not after the caller or the context: `costFormatters.js`, not `renderCostHelpers.js`.
- File names must not include the words `render`, `dom`, `state`, `patch`, `mutate`, or `update` unless the name is an exact match for an existing function group.
- All Wave 1 files must reside in a designated `utils/` or `helpers/` directory that did not previously contain orchestration code. If this directory does not exist, create it. Do not place Wave 1 modules in the same directory as the main application file.

19.2 Function naming

- Extracted functions must retain their original names exactly. No renaming.
- If the original function was anonymous (e.g., an arrow function stored in a variable), it must be exported with the same variable name.
- Do not add prefixes (`pure`, `helper`, `util`) to function names during extraction.

19.3 File-boundary rules

- One logical grouping per file. Constants in one file, formatters in one file, validators in one file. Do not combine categories.
- A Wave 1 file must not exceed the minimal scope of its category. If a formatter file starts to require clinical calculation logic to produce its output, it has crossed a category boundary — split it or reclassify.
- Do not create sub-directories within the Wave 1 output directory during Wave 1. Flat structure only.

19.4 Forbidden naming patterns

- Do not name any file `index.js`, `main.js`, `app.js`, `core.js`, or `engine.js` in Wave 1.
- Do not name any file to suggest it replaces or wraps the main application: `clinicalEngine.js` (reserved for Wave 2+ if `ClinicalEngine` is ever extracted), `stateManager.js` (forbidden until Wave 4+).

---

20. State / Orchestration Safety Boundaries

This section defines the hard boundaries that Wave 1 must never cross. These boundaries are derived directly from the EXTREME and CRITICAL severity items in the architecture risk consolidation.

Boundary B-01: The global state object `S` is untouchable No Wave 1 extraction may read, write, import, export, wrap, proxy, or otherwise reference `S` in any extracted module. `S` may be passed as a parameter to an extracted pure function — the function receives it as a value, does not import it. The module-scoped `S` singleton remains exclusively in the main application file.

Boundary B-02: The tempState shadow is untouchable Same as B-01 for `tempState`. The selective promotion mapping (`whatIfApply` field list) must remain in the main application file exactly as-is.

Boundary B-03: The render orchestration sequence is frozen The sequence `render(S)` → `renderMainPanels()` → `withErrorBoundary` closures → individual panel renders → SVG two-pass is frozen. Wave 1 does not touch this sequence. No helper extracted in Wave 1 may be inserted into this sequence or alter its execution order.

Boundary B-04: The save-render side-effect chain is frozen The chain `setState()` → `saveState()` → 200ms debounce → `renderPatientList()` is frozen. Wave 1 does not touch any function in this chain. No extracted helper may be inserted into or alter this chain.

Boundary B-05: Timer ownership stays in the main application `_typewriterTimer`, `_matFadeTimer`, `_saveStateTimer`, `_notesDebounce`, and the 300ms patient switch timer remain as module-scoped singletons in the main application file. Wave 1 does not extract, wrap, or abstract any timer management.

Boundary B-06: The withErrorBoundary error recovery terminal state is frozen The behavior where an error catch replaces a container with an error fallback and leaves it in that state until a force-rebuild is frozen. Wave 1 does not alter the error recovery path in any way.

Boundary B-07: The observer closure stale-capture behavior is frozen The `_compTableObserver` closes over `state` and `ai` at creation time. This stale capture behavior is existing, expected behavior. Wave 1 does not introduce any mechanism that would refresh these captures or change the observer lifecycle.

Boundary B-08: The two-pass SVG compound highlight is frozen The first pass inside `renderMainPanels()` and the second pass directly in `render()` that overwrites it must remain in their exact positions. Wave 1 does not extract, move, or abstract either pass.

Boundary B-09: Clinical calculation routing is frozen The `MISSING_SINGLE`, `MISSING_MULTI`, `RESTORATIVE` routing inside `ClinicalEngine.process()` is frozen for Wave

1. Individual pure sub-functions within `ClinicalEngine` may be extracted in Tier 4, but the routing logic itself must remain in `ClinicalEngine.process()` in the main application.

Boundary B-10: The patient switch 300ms deferred render is frozen The `setTimeout(() => { render(S); ... }, 300)` inside `switchPatient()` is frozen. The 300ms gap — and the mixed DOM state it produces — is documented existing behavior. Wave 1 does not alter `switchPatient()` in any way.

Appendix A: Wave 1 Extraction Log Template

For each completed extraction, record:

```
Extraction Record
═══════════════════════════════════════════════════

Target name:
Source file and line range (before):
Destination file (after):
Extraction tier:
Call sites modified: (list each)
Pre-extraction checks: (pass/fail for each Section 4 check)
Coupling checks: (findings for each Section 5 check)
Behavioral baseline: (inputs → outputs recorded)
Human review required (Y/N):
Human review decision:
Validation results: (pass/fail for each Section 8 check)
Rollback checkpoints: (before tag, after tag)
Timing delta (if applicable):
Notes / findings during extraction:
```

Appendix B: Forbidden Function Quick Reference

For use during static analysis scans. Any reference to the following in an extraction candidate is an automatic FAIL:

`render`, `renderMainPanels`, `renderTxCards`, `renderPatientDisplay`, `renderComparisonTable`, `lazyRenderComparisonTable`, `renderCompoundSummary`, `renderPatientList`, `renderMaterial`, `renderRisk`, `renderReasons`, `renderCost`, `renderGraph`, `buildAICardStructure`, `updateAICard`, `updateAICardMulti`, `updateCrownCardState`, `updateToothHighlight`, `updateToothHighlight2`, `showSkeleton`, `setState`, `saveState`, `switchPatient`, `selectTx`, `applyAIRec`, `toggleMultiSite`,

`updatePreview`, `debouncedUpdatePreview`, `applyPreset`, `buildEditForm`, `initClinicalNotes`, `updateSliderPositions`, `cancelEdit`, `exitEdit`, `saveEdit`, `generateReport`, `addPatient`, `deletePatient`, `History.add`, `History.render`, `History.save`, `History.load`, `withErrorBoundary`, `typewriterEffect`, `animateNumber`, `showSaveIndicator`, `loadAllPatients`, `saveAllPatients`, `setActivePatientId`, `getActivePatientId`, `genCaseNum`, `closeSidebar`, `updateSiteTabUI`, `switchSite`, `CleanupRegistry`, `_typewriterTimer`, `_matFadeTimer`, `_saveStateTimer`, `_notesDebounce`, `_compTableObserver`, `_lastSimAI`, `_isPreviewMode`, `_notePatient`, `_notesInputBound`, `S.`, `tempState.`, `Object.assign(S`, `Object.assign(tempState`

Document status: Execution governance. Binding for all Wave 1 extraction activity. No architectural changes, refactors, optimizations, or redesigns are contained herein or implied. All rules are derived from documented system behavior.